

In-IDE Toolkit for Developers of AI-Based Features

Yaroslav Sokolov*
JetBrains
Berlin, Germany
yaroslav.sokolov@jetbrains.com

Yury Khudyakov*
JetBrains
Munich, Germany
yury.khudyakov@jetbrains.com

Lenar Sharipov
JetBrains
Munich, Germany
lenar.sharipov@jetbrains.com

Andrei Gasparian
JetBrains
Amsterdam, Netherlands
andrei.gasparian@jetbrains.com

Parth Tiwary
JetBrains
Krakow, Poland
parth.tiwary@jetbrains.com

Artem Trofimov
JetBrains
Berlin, Germany
artem.trofimov@jetbrains.com

Abstract

AI-enabled features built on LLMs and agentic workflows are difficult to test, debug, and reproduce, especially for product-focused software engineers without a machine learning background. We present the *AI Toolkit*¹ plugin for JetBrains IDEs, which brings tracing and evaluation directly into the Run/Debug loop. A mixed methods study with practitioners presents three consistent needs: (1) make evaluation regular and repeatable, (2) expose traces at the moment of execution, and (3) minimize setup and context switching. Guided by these needs, the AI Toolkit introduces an IDE-native workflow: run-triggered trace capture; immediate, hierarchical inspection; one-click “Add to Dataset” from traces; and unit-test-like evaluations with pluggable metrics. The first release in PyCharm shows promising early signals – strong conversion when promoted at Run, sustained usage among those who capture traces, and low churn – suggesting that IDE-native observability lowers activation energy and helps developers adopt disciplined practices. We detail the design and implementation of the AI Agents Debugger and AI Evaluation, report initial adoption telemetry, and outline next steps to broaden framework coverage and scale evaluations. Together, these results indicate that integrating AI observability and evaluation into everyday IDE workflows can make modern AI development accessible to non-ML specialists while preserving software-engineering practices.

CCS Concepts

• **Software and its engineering** → **Integrated and visual development environments**; • **Computing methodologies** → **Machine learning**; **Artificial intelligence**; **Natural language processing**.

Keywords

Integrated Development Environment, Machine Learning, Artificial Intelligence, AI Agents, Natural Language Processing

*Both authors contributed equally to this research.

¹<https://plugins.jetbrains.com/plugin/26921-ai-agents-debugger/>



This work is licensed under a Creative Commons Attribution 4.0 International License. *IDE '26, Rio de Janeiro, Brazil*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2384-1/2026/04

<https://doi.org/10.1145/3786151.3788605>

ACM Reference Format:

Yaroslav Sokolov, Yury Khudyakov, Lenar Sharipov, Andrei Gasparian, Parth Tiwary, and Artem Trofimov. 2026. In-IDE Toolkit for Developers of AI-Based Features. In *3rd International Workshop on Integrated Development Environments (IDE '26)*, April 12–18, 2026, Rio de Janeiro, Brazil. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3786151.3788605>

1 Introduction

AI-enabled features are increasingly built on top of large language models (LLMs) and agentic workflows that orchestrate prompting, retrieval, tools, and memory. While these systems accelerate development, they are often experienced as *black boxes*: behavior depends on data, context, model versions, and non-deterministic sampling, making failures hard to localize and successes hard to reproduce. As teams scale AI features, they need tools for observing agent behavior, reasoning about changes, and verifying that improvements transfer from local tests to real projects.

A growing ecosystem addresses parts of this need, including LangSmith [17], Weights & Biases [3, 31], MLflow [34], and LangFuse [25]. Collectively, these tools provide (i) *tracing* to capture prompts, tool calls, and intermediate states; (ii) *evaluation* to compare variants with human or automated metrics; and (iii) *experiment tracking* to record configurations and outcomes over time. However, these capabilities are typically offered as separate services or dashboards, optimized for ML practitioners who already know which levers to pull and how to stitch them together.

We observe an emerging group of *AI-based feature developers*: product-focused software engineers who add LLM/agent capabilities to existing applications without a background in ML. This cohort is growing rapidly – Gartner projects that by 2027, 55% of software engineering teams will build LLM-based features [10]. To solve day-to-day problems, these developers often adapt familiar monitoring tools rather than adopt new AI-native solutions [26].

There is a clear gap between *AI-native tooling* and *AI-based feature developers*. In our understanding there are several reasons for that: first, these developers lack the intuition that the new development approach is needed for AI systems, including (a) systematic *evaluation* beyond ad hoc testing, (b) structured *experiments* rather than one-off tweaks, and (c) *observability* of the behavior of non-deterministic AI systems. Second, the existing AI-native tooling is optimized for ML developers and is not intuitive for Software Engineers. Even when powerful tools are available, the user journey is unclear: developers are dropped into rich dashboards without

guidance on ‘what to do next’. Crucially, *AI-based feature developers* live in already well established ecosystem [27] which includes IDEs, not in external web consoles. Setting up a context outside the coding environment lowers adoption of existing *AI-native tooling*.

To bridge this gap we propose our AI Toolkit plugin¹ for JetBrains IDEs [14], bringing observability and evaluation workflows into the developer’s primary environment. Our contributions are:

- **User needs research.** Using data from a developer survey conducted by the JetBrains Strategic Research & Market Intelligence team together with our curated interviews, we characterize common pain points that we will address by our tooling.
- **Design and implementation.** We propose an IDE-native AI Toolkit which ties together tracing and evaluation aligned with everyday coding tasks. Then we describe the AI Toolkit plugin architecture and its integration points with project code, run configurations, and test artifacts, enabling low-friction setup and repeatable runs.
- **Release & early signals.** We summarize initial adoption indicators and usage telemetry reflecting whether the IDE-native approach helps developers make evaluation and tracing regular parts of their workflow.

Taken together, our work aims to reduce the cognitive overhead of modern AI development by meeting developers where they work, guiding them from “I just want this agent to behave” to a disciplined loop of tracing and continuous evaluation inside IDE.

2 Developer Needs & Method

To surface developer needs and implications for IDE-native observability and evaluation, we combined data from a developer survey conducted by the JetBrains Strategic Research & Market Intelligence team with our own semi-structured interviews of product-focused engineers building LLM/agent features.

2.1 Survey Insights

The JetBrains Strategic Research & Market Intelligence team recruited respondents through the JetBrains Research Panel in two groups: (i) AI/ML practitioners or employees of AI-active companies (as profiled in the Developer Ecosystem Survey, 2024), and (ii) a general coding audience matching the ecosystem survey on company size, language/IDE usage, and experience. Multi-select questions mean table percentages may exceed 100

Question: What are or were the main challenges you have faced while working on your AI-powered apps or features?

Response option	Share
Difficulty in testing AI features due to non-deterministic behavior	56%
High costs associated with AI services or infrastructure	53%
Data privacy or regulatory constraints	52%
Insufficient quality or accuracy of AI models	50%
Challenges in building robust evaluation pipelines	36%
Complexity or lack of AI-specific tools or frameworks	28%
Other	1%
None	3%

Table 1: Top challenges in AI feature development.

Testing under non-determinism (56%) and model-quality concerns (50%) remain difficult, and teams struggle to assemble robust evaluation pipelines (36%) and AI-specific tooling (28%). While high costs (53%) and data privacy/regulatory issues (52%) are substantial industry concerns, they are largely organizational/infrastructure problems beyond the scope of our IDE plugin; accordingly, we focus on tracing/observability and evaluation.

Question: Which features of the tools or platforms for developing AI agents did you or your company use?

Response option	Share
Standard built-in components (e.g., memory, planning)	62%
Tracing / Observability	48%
Evaluation of AI models’ performance	45%
Advanced debugging (e.g., visual debugging)	32%
I don’t know	9%
Other	2%
None	4%

Table 2: Adopted features in AI agent tooling.

While “built-in components” (62%) dominate, they are framework-native and outside our integration scope. Apart from that, tracing/observability (48%) and evaluation (45%) rank as the most sought-after cross-framework capabilities, and interest in advanced debugging (32%) indicates developers are actively seeking debugging support from tooling. This reinforces the need to surface these features directly in the IDE and to guide their effective use.

Question: How do you evaluate or test the performance of your AI-powered apps or features?

Response option	Share
I manually test AI features	58%
I use user feedback or production metrics	42%
We have dedicated team members (e.g., a team of ML engineers)	32%
I build automated evaluation pipelines using specialized tools	23%
We don’t have a formal evaluation approach in place	10%

Table 3: Evaluation and testing practices.

Developers rarely build automated evaluation pipelines; evaluation is predominantly manual or deferred to production feedback.

Implications. These findings validate our problem framing: teams lack clear, low-friction paths to systematic evaluation; an IDE-native workflow that unifies tracing and evaluation can meet developers where they work and convert ad hoc testing into repeatable practice.

2.2 Interview Insight

We conducted follow-up semi-structured interviews with eight engineers who build AI-powered applications. In each session, participants walked us through how they constructed their systems and how they determined when an agent’s output is “good enough,” with a focus on pre-release evaluation practices.

The core insight is that, without an ML background, pre-release evaluation is often absent: practitioners ship prototypes after manual self-checks and then depend on production metrics and user feedback to assess quality. Elaborating on this, LLM-based development decouples evaluation from training, so teams frequently skip systematic evaluation entirely. Lacking ML intuitions (e.g., datasets,

metrics, and experiment structure), they default to manual testing and in-production signals.

2.3 Design Requirements

Combining our mixed-methods findings, the survey highlights difficulty testing non-deterministic behavior (Table 1), strong demand for tracing/observability and evaluation (Table 2), and a reliance on manual testing or production feedback over automated pipelines (Table 3). Interviews further show that non-ML engineers often skip pre-release evaluation altogether. We therefore articulate the following design requirements:

- DR1 **Prioritize tracing and evaluation.** In the IDE-native toolset we are building, tracing/observability and evaluation must be high-priority capabilities.
- DR2 **Datasetless start.** Evaluation should naturally incorporate data from manual testing and production signals; it must not require a pre-existing dataset.
- DR3 **Education.** We should educate through evaluation (e.g., through lightweight guidance or read-to-use templates)

3 Design and Implementation

We present the AI Toolkit plugin¹ for PyCharm [14], consisting of two main components: AI Agents Debugger and AI Evaluation, aligning with DR1. The former auto-captures and visualizes traces of LLM/agent launches at Run/Debug; the latter turns examples into datasets and runs repeatable evaluations with pluggable metrics.

Our approach is to surface AI observability and evaluation directly at the developer’s *Run* moment within the IDE, turning an ad hoc cycle (“try a prompt, look at the output”) into a tight loop of *trace* → *inspect* → *curate to dataset* → *evaluate* – with near-zero setup and without switching to external dashboards.

In order to achieve high adoption, we need to make it an “easy journey” for the users. Thus, we optimize for three requirements:

- UX1 **Precise target action:** Propose value at the exact moment when a user needs it and has time for it.
- UX2 **Clarity of value:** Show immediate, visible results.
- UX3 **Low activation energy:** Require minimum configuration, avoid code edits and external accounts for a useful baseline.

3.1 Tracing and Observability in the Run Loop

Modern tracing tools (e.g., LangSmith [17], LangFuse [25], and Weights & Biases [3, 31]) capture prompts, tool calls, and intermediate states, but they typically assume outside-the-IDE workflows. We instead bind tracing to the IDE’s *Run* action, so that observability appears precisely when code executes. To cover the broader agentic Python ecosystem, we support the major agentic framework, LangChain [6], including LangGraph [16].

Our tracer uses a client–server architecture. At runtime, a lightweight client, embedded through a Python wrapper, intercepts framework events via callbacks, serializes them, and streams them to an IDE-side server over TCP. The server deserializes, processes, and renders the events in a tool window (Figure 1, step 1.1). When available, the client also emits the agent’s execution graph, which the server renders in a dedicated *Graph* tab (Figure 1, step 1.2).

Tracing is enabled only for projects that the plugin classifies as *AI projects*. The plugin indexes project files for imports of `langchain` or `langgraph`; if any are found, it marks the project accordingly.

When the user clicks **Run** or **Debug** (Figure 1, step 1), the plugin starts the local server and launches the user’s code under the Python wrapper. The wrapper establishes the TCP channel and attaches a `LangChain CallbackHandler` [18] to each `Runnable` [19]. The handler captures events (node entry/exit, LLM and tool calls, and other events), serializes them, and streams them to the server.

The tool window updates incrementally as events arrive, making the tracer compatible with **Debug** execution: the view reflects the run “so far,” enabling step-by-step debugging of agent behavior.

Unlike many existing solutions [17, 25, 31], we default to a *Pretty* view that surfaces only the signal-bearing fields of each event. In Figure 1, for example, the first agent node shows only the invoked tool (`read_schedule`), while the second agent node shows only the LLM’s last message. This preserves a readable narrative of the run without the verbosity of full callback payloads.

For each node, the tracer displays execution time and token usage, enabling quick profiling of latency and cost.

This design satisfies UX1, UX2, and UX3. For UX1, the tracer appears exactly when the user clicks **Run**, and it activates only in projects that already depend on `LangChain` [6] or `LangGraph` [16]. For UX2, we show immediate, visible results: the tracer tool window appears automatically, and it updates dynamically during the agent’s run; it also displays the agent’s execution graph, helping visualize the code-defined structure. For UX3, configuration is minimal – the plugin wraps the user’s code automatically and wires callbacks to the frameworks – keeping activation energy low.

3.2 Evaluation: Unit-tests-like for AI

To make evaluation habitual for software engineers, we model it after unit tests: developers declare data points as test cases (no hand-written evaluation loops) and run them on the current code version without code changes. In this model, a *dataset* is a table of inputs with reference outputs; an *evaluation* executes the user’s code/agent over each input, collects the generated outputs, and scores them with a configurable set of *evaluators*. Evaluators include standard metrics, LLM-as-a-judge [11], and lightweight custom code that returns a run score (see step 4 in Figure 1).

From any captured trace in the AI Agents Debugger described earlier, developers can promote the example directly into the dataset: inputs are auto-extracted while reference outputs remain editable, closing the manual-testing loop and providing a low-friction curation path – thus fulfilling DR2 (see Figure 1, step 1.3).

AI Toolkit then runs the user’s workflow through any in-IDE run configuration, chosen during the Evaluation Configuration setup (Figure 1, step 5.1). Under the hood, we instrument the Python process at launch – monkey-patching the relevant I/O boundaries – to propagate dataset inputs into the system without code edits, aligning with UX3. The AI Toolkit captures the run’s trace, and we reapply the output JSON path specified during dataset curation (Figure 1, step 2.1) to automatically extract the generated outputs.

The results are presented in a bottom tool window analogous to a unit-test runner (Figure 1, step 4), where each datapoint appears as a test case. Developers can inspect the generated output, evaluator

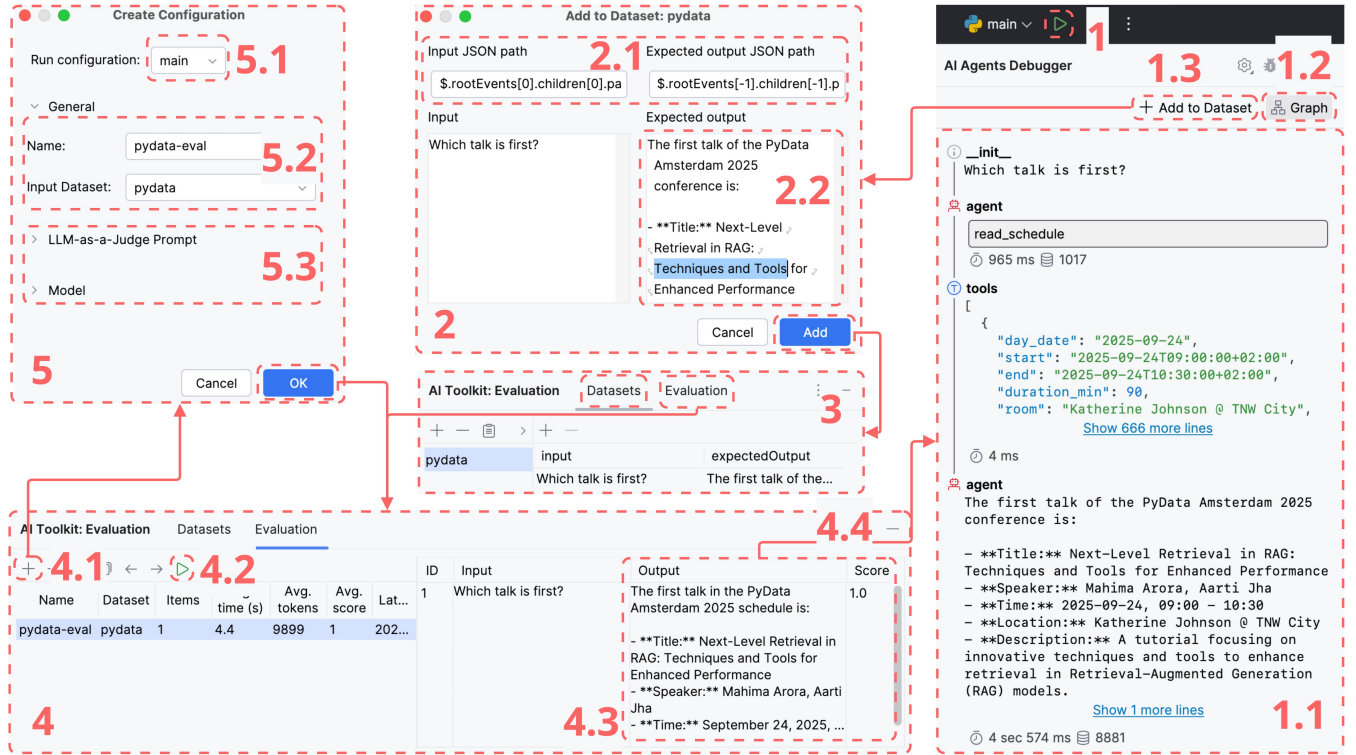


Figure 1: AI Toolkit observe-evaluate flow.

scores, and – when using LLM-as-a-judge [11] – the optional explanation. Aggregated statistics such as average score, token usage, and wall-clock time are also summarized.

We persist the complete trace per data point so that when an output is undesired, the developer can look at the internal calls and prompts to understand what happened (Figure 1, step 4.4).

We store datasets and artifacts as project files (YAML files), making them easy to review, diff, and version with standard VCS. This also enables lightweight collaboration by sharing datasets and produced artifacts through version control.

Execution can be offloaded to remote, long-running jobs orchestrated by the Cadence (ex-JetTrain [30]) backend; progress streams into the IDE with resumable logs and artifact sync, supporting parallelism and recovery without leaving the development environment.

Fulfilling UX2 is challenging because users must complete several steps before seeing results (cf. Figure 1). To assist first-time users – and to meet DR3 – we provide a lightweight onboarding flow (not shown in the figure): contextual tooltips, an “Add to dataset” suggestion from a recent trace, prefilled evaluation templates, and a one-click first run that produces an initial score and artifacts.

3.3 End-to-end Flow Overview

Rather than pushing developers to external dashboards, we anchor observability and evaluation to the IDE’s *Run* action, compressing the informal “try–look into” routine into an in-IDE loop of *trace* → *inspect* → *curate to dataset* → *evaluate*, with minimal setup and no context switching, as illustrated in Figure 1:

- 1) Run & explore traces.** The user clicks *Run* to execute the agent (1); the AI Agents Debugger shows the trace hierarchy (1.1); optionally, the agent graph is displayed (1.2).
- 2) Curate dataset from a trace.** User can choose “Add to Dataset” (1.3); then the “Add to Dataset” dialog opens (2); the user configures JSON paths for input and output extraction (2.1) – the same output path is later reused to automatically extract generated outputs during evaluation – and verifies or adjusts the expected output (2.2).
- 3) The “Evaluation > Datasets” view is shown (3), followed by the “Evaluation > Evaluation” view (4).**
- 4) Create Evaluation Configuration.** User clicks the “+” button to start creating an Evaluation Configuration (4.1); the “Create Configuration” window opens (5); the user selects an existing agent Run configuration to execute evaluations (5.1), chooses a name and input dataset (5.2), and configures the LLM-as-a-judge [11] prompt and model (5.3).
- 5) Run Evaluation and inspect.** Back in the Evaluation view (4), the user clicks *Run* to launch the evaluation (4.2); outputs are shown per row (4.3); for any datapoint, the corresponding trace can be inspected in the AI Agents Debugger (4.4).

4 Release Outcomes

We have released AI Toolkit as a separate plugin, which one needs to install to start using it. Then we made a promo campaign in PyCharm [14] for the AI Toolkit following the PyCharm 2025.2

release. At the moment, AI Toolkit only contained AI Agents Debugger functionality with only LangGraph [16] supported. The promo campaign included (i) the popup appearing after *Run* button was clicked in the projects with LangGraph, suggesting to install the AI Toolkit plugin (ii) description of the plugin in “What’s new” tab, shown after PyCharm update.

Reach and interest. In the four weeks following the PyCharm 2025.2 release, we saw 970 new users of the AI Toolkit. The popup window had a pretty high installation rate (58%), and generated a one-third of all new users. The other two-thirds arrived from other sources, mainly the “What’s new” tab.

Activation. About 40% of those who installed via popup, captured a first trace. However, only around 12% of users who installed via “What’s new”, captured a first trace. The main stopper is the frameworks coverage, not intent: most newcomers (particularly those arriving from “What’s New”) did not use LangGraph in the current project, and the released AI Agents Debugger implementation only supported LangGraph-based agents. Users in unsupported setups typically ignore the plugin rather than churn.

Retention and churn. By week four, 26% of all installers remained active; among developers who actually made any runs in PyCharm, week-four retention rises to 40%. Churn (disable/uninstall) remained low at roughly 10–15%.

Takeaways. High conversion at *Run*, solid week-four retention, and low churn indicate that the IDE-native, run-triggered workflow is legible and valuable. The principal opportunity is to broaden the framework support (beyond LangGraph).

5 Next Steps & Limitations

Next steps.

- **Broader framework coverage.** Early indicators suggest that activation will be capped by framework support rather than developer intent. To reach a broader user base, we will release an updated plugin with planned support for the Kotlin agentic framework *Koog* [13], *PydanticAI* [7, 24] and *OpenAI Python SDK* [23]. Following these releases, we will monitor usage signals and metrics to guide further development and extend support to additional agent stacks as demand evolves.
- **Track the usage of AI Evaluation.** We will monitor the adoption of AI Evaluation by collecting usage statistics. Based on feedback, we will prioritize UX improvements and tighter integration between AI Agents Debugger and AI Evaluation.

Limitations and threats to validity.

- **Narrow initial coverage.** The reported activation reflects a release that supported only *LangGraph*, leaving much of the agentic-framework space uncovered; consequently, true demand is likely under-estimated.
- **Short observation window.** Metrics summarize only the first four weeks of a single release, with a sample of roughly 1,000 users. A larger sample and a longer observation window will yield more precise estimates.

- **PyCharm-only sample bias.** We have released a plugin only for *PyCharm* [14], while a large portion of developers use other IDEs [27], such as *Visual Studio Code* [21].

6 Related work

The rapid development of AI-based applications, particularly those powered by autonomous or semi-autonomous agents, is transforming the software landscape [9, 15, 22, 28]. As these AI agents grow more complex and integrated into diverse workflows, the need for effective tools to monitor, evaluate, and improve their behavior becomes increasingly critical [4, 5, 8, 35].

There are several tools that provide agent observability features at scale. Among them are LangSmith [17] by LangChain [1], Weights & Biases [3, 31], MLflow [34], and LangFuse [25]. As mentioned earlier, these capabilities are typically offered as separate tools or dashboards, designed for ML practitioners who already know which controls to use and how to integrate them effectively.

Evaluation of AI agents is an active area of research [32, 33, 36] and industry [2, 12, 29]. Research efforts focus on developing general evaluation approaches (e.g., human-based, LLM-as-a-judge [11]), assessing agent capabilities (e.g., planning, function calling), and designing evaluation methods for particular domains (e.g., software engineering agents, data agents) [32]. Industrial solutions emphasize collaboration features and integration with existing customer infrastructure. The main goal of this work is to provide a simple, usable evaluation flow directly within the IDE.

A related initiative is the VS Code AI Toolkit by Microsoft [20]. It follows a similar goal of improving the developer experience for AI workflows but focuses on a different set of capabilities. Specifically, it offers tight integration with Azure AI Studio for model fine-tuning, deployment, and access to a model catalogue, while also including basic evaluation and traceability features. In contrast to our approach, which emphasizes lightweight, IDE-native evaluation and observability, the VSCode AI Toolkit is designed as part of a broader Azure ecosystem that streamlines the connection between local development and cloud-based model management.

7 Conclusion

By meeting developers in the IDE and guiding tracing, evaluation, and experimentation as a single workflow, the AI Toolkit lowers cognitive overhead and makes disciplined AI development accessible to non-ML specialists. This paper contributes: (i) empirical evidence of developer needs around non-determinism and ad hoc testing; (ii) an IDE-native design that ties observability and evaluation to the Run/Debug moment, including run-triggered tracing and a curated path from trace → dataset → evaluation; and (iii) an implementation in JetBrains IDEs (AI Agents Debugger and AI Evaluation) that delivers these capabilities with near-zero setup.

Early usage indicates that this approach is legible and valuable: developers respond to in-context prompts at Run, successfully capture traces, and retain the tool at meaningful rates; the main activation bottleneck is framework coverage rather than intent. While results reflect a short window and an initially narrow scope, they provide actionable direction.

More broadly, we argue that bringing AI observability and evaluation into the IDE – rather than exporting developers to external

dashboards – offers a practical path to AI features in real projects. We invite the IDE and SE communities to build on this pattern: standardize trace schemas, treat evaluations as first-class tests, and connect IDE-native feedback loops to CI and production telemetry.

Acknowledgments

We thank Mikhail Bogdanov from the Survey team of the Strategic Research & Market Intelligence department of JetBrains for designing, collecting, cleaning, and analyzing survey data, and Olga Galchenko, Artem Sarkisov, and Sofia Kulikova for their initiative, for contributing to questionnaire development, and for their domain expertise.

References

- [1] Sangeetha Annam, Merry Saxena, Ujjwal Kaushik, and Shikha Mittal. 2025. LangChain: Simplifying Development with Language Models. *Textual Intelligence: Large Language Models and Their Real-World Applications* (2025), 287–304.
- [2] Arize AI, Inc. 2025. Arize AI. <https://arize.com>. AI & Agent Engineering Platform: development, observability, and evaluation. Accessed: 2025-10-27.
- [3] Lukas Biewald. 2020. Experiment Tracking with Weights and Biases. <https://www.wandb.com/>. Software available from <https://wandb.com>. Accessed: 2025-10-27.
- [4] Alan Chan, Carson Ezell, Max Kaufmann, Kevin Wei, Lewis Hammond, Herbie Bradley, Emma Bluemke, Nitarshan Rajkumar, David Krueger, Noam Kolt, et al. 2024. Visibility into AI agents. In *Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency*. 958–973.
- [5] Alan Chan, Kevin Wei, Sihao Huang, Nitarshan Rajkumar, Elia Perrier, Seth Lazar, Gillian K. Hadfield, and Markus Anderljung. 2025. Infrastructure for ai agents. <https://arxiv.org/abs/2501.10114>. arXiv:2501.10114.
- [6] Harrison Chase. 2022. LangChain. <https://github.com/langchain-ai/langchain>. Open-source Python framework for developing applications powered by language models. Accessed: 2025-10-27.
- [7] Samuel Colvin, Eric Jolibois, Hasan Ramezani, Adrian Garcia Badaracco, Terrence Dorsey, David Montague, Serge Matveenko, Marcelo Trylesinski, Sydney Runkle, David Hewitt, Alex Hall, and Victorien Plot. 2025. Pydantic Validation. <https://docs.pydantic.dev/latest/>. Version v2.13.0a0+dev. Widely used data validation library for Python 3.9+ with strong typing and JSON Schema support. Source: <https://github.com/pydantic/pydantic>. License: MIT. Accessed: 2025-10-27.
- [8] Liming Dong, Qinghua Lu, and Liming Zhu. 2024. AgentOps: Enabling Observability of LLM Agents. <https://arxiv.org/abs/2411.05285>. arXiv:2411.05285.
- [9] Shanghua Gao, Ada Fang, Yepeng Huang, Valentina Giunchiglia, Ayush Noori, Jonathan Richard Schwarz, Yasha Ektefaie, Jovana Kondic, and Marinka Zitnik. 2024. Empowering biomedical discovery with AI agents. *Cell* 187, 22 (2024), 6125–6151.
- [10] Gartner, Inc. 2025. Gartner Identifies the Top Strategic Trends in Software Engineering for 2025 and Beyond. <https://www.gartner.com/en/newsroom/press-releases/2025-07-01-gartner-identifies-the-top-strategic-trends-in-software-engineering-for-2025-and-beyond>. Press release. Contains the prediction that by 2027, at least 55% of software engineering teams will be actively building LLM-based features. Accessed: 2025-10-27.
- [11] Jiawei Gu, Jianqing Zhao, Chenghao Liu, Wenxin Du, Zihuiwen Fang, Yao Wang, Wenqiang Lei, and Wayne Xin Zhao. 2024. A Survey on LLM-as-a-Judge. <https://arxiv.org/abs/2411.15594>. arXiv:2411.15594. Survey of LLMs as judges for generative evaluation, with techniques to improve reliability and consistency. Accessed: 2025-10-27.
- [12] Jeffrey Ip and Kritin Vongthongsri. 2025. deepeval. <https://github.com/confident-ai/deepeval>. Version 3.6.8. License: Apache-2.0. Accessed: 2025-10-27.
- [13] JetBrains. 2024. Koog by JetBrains. <https://www.jetbrains.com/koog/>. Open-source framework for building fault-tolerant, scalable, production-ready AI agents with a Kotlin DSL; graph-based workflows; built-in tracing/monitoring; integrates with Weave and Langfuse. Accessed: 2025-10-27.
- [14] JetBrains. 2025. PyCharm: The Only Python IDE You Need. <https://www.jetbrains.com/pycharm/>. Professional Python IDE with AI-enhanced experience; supports Python, databases, Jupyter, Git, Conda, and frameworks incl. PyTorch, TensorFlow, Django, Flask, FastAPI. Accessed: 2025-10-27.
- [15] Hyoungwook Jin, Seonghee Lee, Hyungyu Shin, and Juho Kim. 2024. Teach ai how to code: Using large language models as teachable agents for programming education. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. 1–28.
- [16] LangChain Inc. 2024. LangGraph. <https://github.com/langchain-ai/langgraph>. Low-level orchestration framework and runtime for building long-running, stateful agents with durable execution, HITL, memory, and production deployment. Accessed: 2025-10-27.
- [17] LangChain Inc. 2024. LangSmith. <https://smith.langchain.com>. Tooling for developing, debugging, evaluating, and deploying LLM applications; integrates with LangChain and LangGraph. Accessed: 2025-10-27.
- [18] LangChain Inc. 2025. LangChain Callback System. <https://python.langchain.com/docs/concepts/callbacks>. Callback system for hooking into LLM app stages; handlers receive start/end events, tokens, errors; pass callbacks per-request or at construction. Accessed: 2025-10-27.
- [19] LangChain Inc. 2025. LangChain Runnable. https://python.langchain.com/api_reference/core/runnables/langchain_core.runnables.base.Runnable.html. Runnable = unit of work that can be invoked, batched, streamed, transformed, and composed; LCEL primitives: RunnableSequence, RunnableParallel. Accessed: 2025-10-27.
- [20] Microsoft. 2025. AI Toolkit for Visual Studio Code. <https://github.com/microsoft/vscode-ai-toolkit>. Accessed: 2025-10-27.
- [21] Microsoft. 2025. Visual Studio Code. <https://code.visualstudio.com/>. Accessed: 2025-10-27.
- [22] Liangbo Ning, Ziran Liang, Zhuohang Jiang, Hao hao Qu, Yujuan Ding, Wenqi Fan, Xiao-yong Wei, Shanru Lin, Hui Liu, Philip S. Yu, et al. 2025. A survey of webagents: Towards next-generation ai agents for web automation with large foundation models. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*. 6140–6150.
- [23] OpenAI. 2025. OpenAI Python SDK. <https://github.com/openai/openai-python>. Official Python SDK for the OpenAI API with typed sync/async clients; supports the Responses and Chat Completions APIs, vision, and more. Accessed: 2025-10-27.
- [24] Pydantic Team. 2025. Pydantic AI. <https://github.com/pydantic/pydantic-ai>. Framework for structured, type-safe, debuggable AI applications with agentic workflows and function calling. Accessed: 2025-10-27.
- [25] Clemens Rawert, Marc Klengen, and Maximilian Deichmann. 2023. Langfuse – Open-Source LLM Engineering Platform. <https://langfuse.com/>. Software available from <https://langfuse.com>. Accessed: 2025-10-27.
- [26] Stack Overflow. 2025. Stack Overflow Developer Survey 2025 – AI agent observability & security. <https://survey.stackoverflow.co/2025/ai#ai-agent-observability-and-security>. Accessed: 2025-10-27.
- [27] Stack Overflow. 2025. Stack Overflow Developer Survey 2025 – Technology: Dev IDEs (Professionals that use AI). <https://survey.stackoverflow.co/2025/technology#most-popular-technologies-dev-envs-prof-ai>. Accessed: 2025-10-27.
- [28] Guangzhi Sun, Xiao Zhan, and Jose Such. 2024. Building better ai agents: A provocation on the utilisation of persona in llm-based conversational agents. In *Proceedings of the 6th ACM Conference on Conversational User Interfaces*. 1–6.
- [29] SuperAnnotate AI, Inc. 2025. SuperAnnotate. <https://www.superannotate.com>. Build efficient human data and evaluation pipelines to ship better agentic, multimodal, and frontier AI faster. Accessed: 2025-10-27.
- [30] Artem Trofimov, Mikhail Kostyukov, Sergei Ugdyzhkov, Natalia Ponomareva, Igor Naumov, and Maksim Melekhovets. 2024. JetTrain: IDE-Native Machine Learning Experiments. In *Proceedings of the 1st ACM/IEEE Workshop on Integrated Development Environments*. 59–61.
- [31] Weights & Biases. 2024. Weave: Track, Test and Improve LLM Applications. <https://wandb.ai/site/products/weave>. Track LLM calls in production, test prompts, and run evaluations for performance and safety. Accessed: 2025-10-27.
- [32] Asaf Yehudai, Lilach Eden, Alan Li, Guy Uziel, Yilun Zhao, Roy Bar-Haim, Arman Cohan, and Michal Shmueli-Scheuer. 2025. Survey on evaluation of LLM-based agents. <https://arxiv.org/abs/2503.16416>. arXiv:2503.16416.
- [33] Guoli Yin, Haoping Bai, Shuang Ma, Feng Nan, Yanchao Sun, Zhaoyang Xu, Shen Ma, Jiarui Lu, Xiang Kong, Anon Zhang, et al. 2025. MMAU: A Holistic Benchmark of Agent Capabilities Across Diverse Domains. In *Findings of the Association for Computational Linguistics: NAACL 2025*. 4737–4765.
- [34] Matei A. Zaharia, Andrew Chen, Aaron Davidson, Ali Ghodsi, Sue Ann Hong, Andy Konwinski, Siddharth Murching, Tomas Nykodym, Paul Ogilvie, Mani Parkhe, Fen Xie, and Corey Zumar. 2018. Accelerating the Machine Learning Lifecycle with MLflow. *IEEE Data Engineering Bulletin* 41 (2018), 39–45. MLflow is an open-source platform for managing the ML lifecycle: tracking, packaging, registry, serving, evaluation, and observability. Available at <https://mlflow.org>. Accessed: 2025-10-27.
- [35] Yusheng Zheng, Yanpeng Hu, Tong Yu, and Andi Quinn. 2025. AgentSight: System-Level Observability for AI Agents Using eBPF. In *Proceedings of the 4th Workshop on Practical Adoption Challenges of ML for Systems*. 110–115.
- [36] Mingchen Zhuge, Changsheng Zhao, Dylan R. Ashley, Wenyi Wang, Dmitrii Khizbullin, Yunyang Xiong, Zechun Liu, Ernie Chang, Raghuraman Krishnamoorthi, Yuandong Tian, et al. 2025. Agent-as-a-Judge: Evaluate Agents with Agents. In *Forty-second International Conference on Machine Learning*.